🐘 Big Data i ekosystem Hadoop

Jak przechowywać i przetwarzać dane, które nie mieszczą się na jednym dysku

Python w Zastosowaniach Big Data | Wykład 3

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Co to jest Big Data i dlaczego "zwykłe" narzędzia się poddają
2. 5V Big Data — definicja, którą musisz znać
3. HDFS — rozproszony system plików
4. MapReduce — paradygmat map/shuffle/reduce (na przykładzie word count)
5. YARN — zarządzanie zasobami klastra
6. Dlaczego Spark zastąpił MapReduce
7. Ekosystem Hadoop — mapa narzędzi

> 🎯 **Po tym wykładzie:** Zrozumiesz, jak Hadoop rozwiązał problem "za dużo danych na jeden komputer" — i dlaczego to wciąż fundament dzisiejszego Big Data.

🔄 Szybkie przypomnienie

Na poprzednim wykładzie (Wykład 2) zobaczyliśmy, jak wycisnąć maksimum z **jednej maszyny** — Parquet zamiast CSV, kompresja, partycjonowanie i równoległe przetwarzanie na wszystkich rdzeniach.

Dziś zmieniamy skalę: co zrobić, gdy dane to **petabajty**, a nawet najmocniejszy serwer fizycznie nie da rady?

> 💡 Scale up ma swój sufit. Czas na **scale out** — klaster maszyn! 🐘

💡 Dlaczego to ważne?

Czy wiesz, że **YouTube** ma ponad **500 godzin wideo** wgrywanych **co minutę**? 🤯

To ponad **720 000 godzin dziennie**. Żaden pojedynczy dysk, żadna pojedyncza baza danych tego nie ogarnie.

Firma	Skala danych	Problem
YouTube	500 h wideo/min	Storage + transkodowanie
Facebook	~4 PB nowych danych/dzień	Analiza w czasie ~real-time
Amazon	Miliardy transakcji/dzień	Rekomendacje na żywo

> To właśnie dla takich problemów powstał **Hadoop**.

😱 Problem ze skalą

Wyobraź sobie plik **10 TB**. Chcesz go przeanalizować w Pythonie:

```
with open("dane.csv") as f:
    for line in f:
        process(line)  # 👈 Ile to będzie trwać?
```

Problemy:

- ⏱️ Odczyt z jednego dysku = **wąskie gardło**
- 💾 Plik może nie zmieścić się na jednym serwerze
- 💥 Serwer padnie w połowie przetwarzania → tracisz wszystko
- 🐌 Jeden CPU przetwarza dane **sekwencyjnie**

> 🤔 **Pytanie:** Co jeśli zamiast jednego potężnego serwera użyjemy **1000 zwykłych komputerów**?

🐘 Co to jest Big Data?

Big Data to dane, które są zbyt **duże**, zbyt **szybkie** lub zbyt **różnorodne**, żeby przetworzyć je tradycyjnymi narzędziami (np. jedną bazą SQL na jednym serwerze).

To nie tylko "dużo danych" — to sytuacja, w której **sposób** przechowywania i przetwarzania danych musi się zmienić.

> 💡 **Analogia:** Excel działa świetnie na 10 000 wierszy. Na 10 miliardach wierszy — Excel się poddaje. Potrzebujesz innej architektury.

🔑 5V Big Data

V	Nazwa	Opis	Przykład
Volume	Wolumen	Ogromna ilość danych	Facebook: ~4 PB/dzień
Velocity	Prędkość	Dane napływają w czasie rzeczywistym	Twitter/X: tysiące postów/s
Variety	Różnorodność	Strukturalne, pół- i niestrukturalne	CSV, JSON, logi, wideo, zdjęcia
Veracity	Wiarygodność	Dane bywają błędne, niekompletne	Brakujące wartości, duplikaty, spam
Value	Wartość	Dane bez insightu są bezużyteczne	+15% sprzedaży dzięki rekomendacjom

> 🎯 Zapamiętaj **5V** — to pojawi się na egzaminie! 😉

📦 Volume — kiedy dane nie mieszczą się na dysku

- Pojedynczy dysk SSD: ~2-4 TB
- Netflix przechowuje **setki petabajtów** danych o oglądalności
- Rozwiązanie: **podziel dane na kawałki** i rozłóż na wiele maszyn

```
# Zamiast: jeden plik na jednym dysku
dane = wczytaj("wszystko.csv")  # 💥 OutOfMemoryError

# Big Data podejście: podziel dane na bloki
# blok_1.csv -> serwer A
# blok_2.csv -> serwer B
# blok_3.csv -> serwer C  (i tak dalej...)
```

> 💡 To jest właśnie idea, na której zbudowany jest **HDFS** — zaraz o nim opowiemy.

⚡ Velocity — kiedy dane napływają szybciej niż zdążysz je zapisać

Przykład: czujniki IoT w fabryce wysyłają odczyt **co sekundę**, z tysiąca maszyn.

- Batch processing (wsadowe) — analizujemy dane **po fakcie**, np. raz dziennie
- Stream processing (strumieniowe) — analizujemy dane **na bieżąco**, w locie

> 🔮 Do przetwarzania strumieniowego wrócimy na Wykładzie 7 (Spark Streaming) — dziś skupiamy się na batch processing, bo to fundament, od którego wszystko się zaczęło.

🎭 Variety — różnorodność formatów

Typ danych	Przykład	Struktura
Strukturalne	Tabela SQL, CSV	Ściśle zdefiniowana (kolumny, typy)
Pół-strukturalne	JSON, XML, logi	Elastyczna, ale ma jakiś format
Niestrukturalne	Zdjęcia, wideo, tekst	Brak z góry narzuconej struktury

```
{
  "user_id": 42,
  "event": "click",
  "metadata": {"button": "buy_now", "device": "mobile"}
}
```

> 💡 Tradycyjne bazy SQL uwielbiają dane strukturalne. Hadoop **nie przejmuje się** formatem — najpierw zapisuje, strukturę nadajesz później ("schema-on-read").

🎯 Veracity — czy możesz ufać swoim danym?

- Czujnik chwilowo się zawiesił → brakujący odczyt
- Użytkownik wpisał literówkę w formularzu → błędny rekord
- Bot wygenerował 10 000 fałszywych kliknięć → szum w danych

> ⚠️ **Garbage in, garbage out.** Nawet najlepszy klaster Hadoop nie naprawi złej jakości danych — trzeba je czyścić (o czym była mowa przy statystyce opisowej!).

💰 Value — po co to wszystko?

Dane same w sobie **nic nie znaczą**. Liczy się **insight**, który z nich wyciągniesz.

- Spotify: analiza miliardów odtworzeń → **Discover Weekly**
- Amazon: analiza koszyków → **"klienci którzy kupili X, kupili też Y"**
- Szpital: analiza danych pacjentów → **wczesne wykrywanie chorób**

> 🎯 **Value to cel.** Volume, Velocity, Variety i Veracity to tylko przeszkody, które musimy pokonać, żeby do niego dotrzeć.

🤔 Think About It

Pytanie: Firma X ma bazę danych 50 GB, która rośnie o 1 MB dziennie i zawsze ma ten sam, czysty format (CSV z 10 kolumnami).

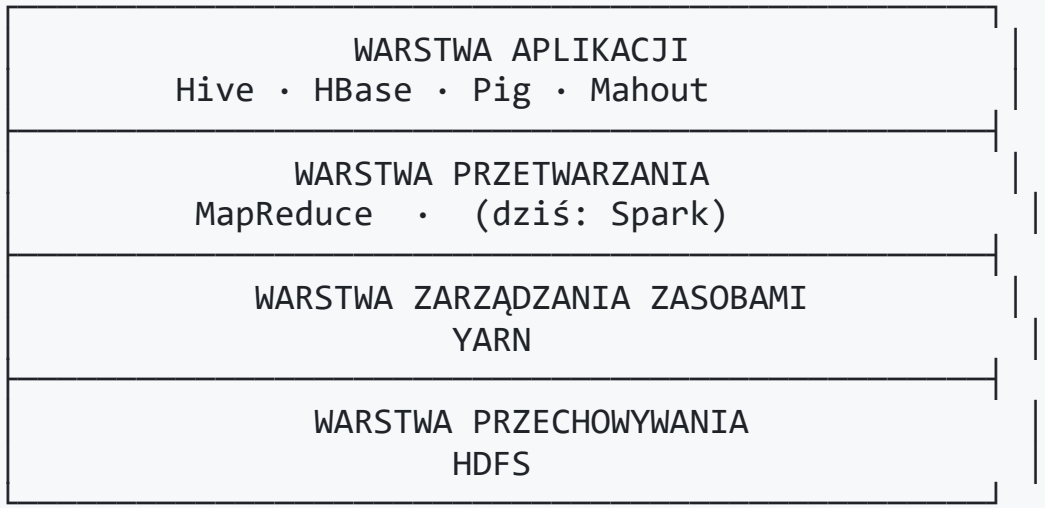
Czy firma X potrzebuje Hadoopa?

📜 Skąd się wziął Hadoop?

- **2003-2004**: Google publikuje papers o **GFS** (Google File System) i **MapReduce**
- **2006**: Doug Cutting i Mike Cafarella (projekt Nutch) tworzą **Apache Hadoop** — otwartą implementację tych idei
- **2008**: Yahoo! uruchamia klaster Hadoop na **10 000+ węzłów**
- Dziś: Hadoop to fundament większości systemów Big Data (nawet jeśli sam MapReduce już rzadziej używany)

> 💡 Hadoop = "wzięliśmy pomysł Google i daliśmy go całemu światu za darmo" 🎁

Ekosystem Hadoop — mapa ogólna



> 🎯 Trzy filary klasycznego Hadoopa: **HDFS** (storage) + **MapReduce** (processing) + **YARN** (resource management).

💾 HDFS — Hadoop Distributed File System

HDFS to rozproszony system plików — dzieli duże pliki na **bloki** i rozkłada je na wiele maszyn w klastrze.

Kluczowe cechy:

- 📦 Domyślny rozmiar bloku: **128 MB** (kiedyś 64 MB)
- 🔁 Każdy blok jest **replikowany** (domyślnie **3 kopie**)
- 🖥️ Zaprojektowany dla **zwykłego, taniego sprzętu** (commodity hardware)
- ✅ Optymalizowany pod **duże pliki, sekwencyjny odczyt** — nie pod małe, częste zapisy

> 💡 Analogia: zamiast trzymać całą encyklopedię w jednym segregatorze, dzielisz ją na tomy i dajesz każdy tom **trzem** różnym bibliotekarzom — jeśli jeden zgubi tom, masz zapasowe kopie.

HDFS — architektura

Komponent	Rola
NameNode	"Mózg" klastra — trzyma metadane (gdzie leżą bloki), ale **nie** trzyma samych danych
DataNode	"Mięśnie" klastra — fizycznie przechowuje bloki danych, raportuje do NameNode
Secondary NameNode	Pomocnik NameNode — robi okresowe kopie metadanych (**nie** jest backupem awaryjnym!)
Client	Aplikacja/użytkownik, który czyta/zapisuje pliki w HDFS



🔁 Replikacja bloków — dlaczego to takie ważne

```
Blok A (128 MB)
  ├── Kopia 1 → DataNode 7   (ta sama szafa serwerowa - "rack")
  ├── Kopia 2 → DataNode 12  (inna szafa - odporność na awarię szafy)
  └── Kopia 3 → DataNode 19  (inne centrum danych, jeśli możliwe)
```

- Domyślny **replication factor = 3**
- Jeśli DataNode 7 padnie → NameNode wykrywa brak "heartbeat" → tworzy nową kopię z DataNode 12 lub 19
- Klaster **sam się naprawia** — to nazywamy **fault tolerance** (odporność na awarie)

> ⚠️ Więcej replik = więcej bezpieczeństwa, ale też więcej zużytej przestrzeni dyskowej. Trade-off!

🤔 Think About It

Pytanie: Dlaczego replikacja na 3 różnych **szafach serwerowych (racks)** jest lepsza niż 3 kopie w tej samej szafie?

⚙️ MapReduce — paradygmat programowania

MapReduce to model przetwarzania danych rozłożony na dwie proste fazy:

1. **Map** — przetwarzasz każdy rekord **niezależnie**, produkując pary (klucz, wartość)
2. **Reduce** — grupujesz wartości po kluczu i **agregujesz** je

Między nimi dzieje się **Shuffle** — Hadoop sam grupuje wszystkie wartości o tym samym kluczu, tak żeby trafiły do tego samego reducera.

```
  MAP          →     SHUFFLE       →     REDUCE
(równolegle,          (grupowanie          (agregacja
 na wielu             po kluczu,            per klucz)
 maszynach)           sortowanie)
```

> 💡 Kluczowa idea: **map działa równolegle na każdym fragmencie danych osobno** — to jest źródło całej mocy MapReduce!

📝 Word Count — klasyczny przykład MapReduce

Zadanie: policz, ile razy każde słowo występuje w zbiorze dokumentów.

Wejście (2 "dokumenty" rozłożone na różne maszyny):

```
Dokument 1: "Python jest super Python jest proste"
Dokument 2: "Spark jest szybki Python jest potężny"
```

Chcemy dostać: `{"Python": 3, "jest": 4, "super": 1, "proste": 1, "Spark": 1, "szybki": 1, "potężny": 1}`

Zobaczmy, jak MapReduce dochodzi do tego wyniku krok po kroku 👇

🗺️ Word Count — faza Map

Każda maszyna dostaje **fragment** danych i **niezależnie** emituje pary `(słowo, 1)`:

```python
def map_function(dokument: str) -> list[tuple[str, int]]:
    """Faza Map: dla każdego słowa emitujemy (słowo, 1)."""
    return [(slowo, 1) for slowo in dokument.lower().split()]

# Maszyna A przetwarza Dokument 1
map_function("Python jest super Python jest proste")
# [("python", 1), ("jest", 1), ("super", 1), ("python", 1), ("jest", 1), ("proste", 1)]

# Maszyna B przetwarza Dokument 2 — RÓWNOLEGLE, w tym samym czasie!
map_function("Spark jest szybki Python jest potężny")
# [("spark", 1), ("jest", 1), ("szybki", 1), ("python", 1), ("jest", 1), ("potężny", 1)]
```

> 🎯 Obie maszyny pracują **jednocześnie** — to jest źródło przyspieszenia!

🔀 Word Count — faza Shuffle

Framework **grupuje** wszystkie pary o tym samym kluczu (słowie) razem — niezależnie od tego, która maszyna je wyprodukowała:

```
# Wszystkie wyniki z fazy Map, połączone:
wszystkie_pary = [
    ("python", 1), ("jest", 1), ("super", 1), ("python", 1),
    ("jest", 1), ("proste", 1), ("spark", 1), ("jest", 1),
    ("szybki", 1), ("python", 1), ("jest", 1), ("potężny", 1),
]

# Shuffle: grupujemy po kluczu
# "python" -> [1, 1, 1]
# "jest"   -> [1, 1, 1, 1]
# "super"  -> [1]
# "proste" -> [1]
# "spark"  -> [1]
# "szybki" -> [1]
# "potężny"-> [1]
```

> 💡 To właśnie ten krok jest **najdroższy** w klasycznym MapReduce — wymaga przesłania danych między maszynami przez sieć i dysk.

➕ Word Count — faza Reduce

Dla każdego klucza sumujemy listę wartości:

```
def reduce_function(slowo: str, wartosci: list[int]) -> tuple[str, int]:
    """Faza Reduce: sumujemy wystąpienia danego słowa."""
    return (slowo, sum(wartosci))

reduce_function("python", [1, 1, 1])   # ("python", 3)
reduce_function("jest", [1, 1, 1, 1])  # ("jest", 4)
reduce_function("super", [1])          # ("super", 1)
```

Wynik końcowy:

```
{"python": 3, "jest": 4, "super": 1, "proste": 1,
 "spark": 1, "szybki": 1, "potężny": 1}
```

> 🎉 Dokładnie to, czego oczekiwaliśmy! I każdy krok **map** dało się wykonać na osobnej maszynie.

💻 Word Count — pełna symulacja w czystym Pythonie

```
from collections import defaultdict

def map_reduce_word_count(dokumenty: list[str]) -> dict[str, int]:
    """Symulacja MapReduce word count – bez klastra, na jednej maszynie."""
    # --- MAP --- (w prawdziwym Hadoopie: równolegle, na wielu maszynach)
    mapped: list[tuple[str, int]] = []
    for dokument in dokumenty:
        mapped.extend((slowo, 1) for slowo in dokument.lower().split())

    # --- SHUFFLE --- (grupowanie po kluczu)
    grouped: dict[str, list[int]] = defaultdict(list)
    for slowo, wartosc in mapped:
        grouped[slowo].append(wartosc)

    # --- REDUCE --- (agregacja per klucz)
    return {slowo: sum(wartosci) for slowo, wartosci in grouped.items()}

dokumenty = ["Python jest super Python jest proste",
             "Spark jest szybki Python jest potężny"]
print(map_reduce_word_count(dokumenty))
# {'python': 3, 'jest': 4, 'super': 1, 'proste': 1, 'spark': 1, 'szybki': 1, 'potężny': 1}
```

🐌 Dlaczego "klasyczny" MapReduce jest wolny?

Po **każdej** fazie map i reduce, wyniki pośrednie są **zapisywane na dysk** (nie w RAM):

```
Map → zapis na dysk → Shuffle (sieć + dysk) → zapis na dysk → Reduce → zapis na dysk
```

- 💾 Odczyt/zapis z dysku jest **~100× wolniejszy** niż z RAM
- 🔁 Zadania iteracyjne (np. Machine Learning, wiele przebiegów po tych samych danych) muszą **za każdym razem** czytać/pisać na dysk od nowa
- ⏱️ Dobre dla **jednorazowego** przetworzenia ogromnych danych wsadowo, słabe dla **iteracyjnych** algorytmów

> 🔮 To jest dokładnie problem, który rozwiąże **Apache Spark** — o czym za chwilę!

⚠️ Częste pułapki z MapReduce

Pułapka	Dlaczego to problem	Co zamiast tego
Zbyt małe pliki wejściowe	NameNode musi trzymać metadane dla każdego pliku — miliony małych plików go zabijają	Łącz małe pliki w większe (np. Parquet, sekwencje)
Zbyt duży `reduce` na jednym kluczu	Jeden reducer dostaje 90% danych (tzw. "data skew")	Lepsze partycjonowanie klucza, salting
Ignorowanie kosztu Shuffle	Programista pisze wiele niepotrzebnych faz MapReduce	Minimalizuj liczbę przejść przez dane
Traktowanie MapReduce jako uniwersalnego narzędzia	Świetne dla batch, słabe dla iteracji i ML	Użyj Spark dla przetwarzania iteracyjnego

YARN — Yet Another Resource Negotiator

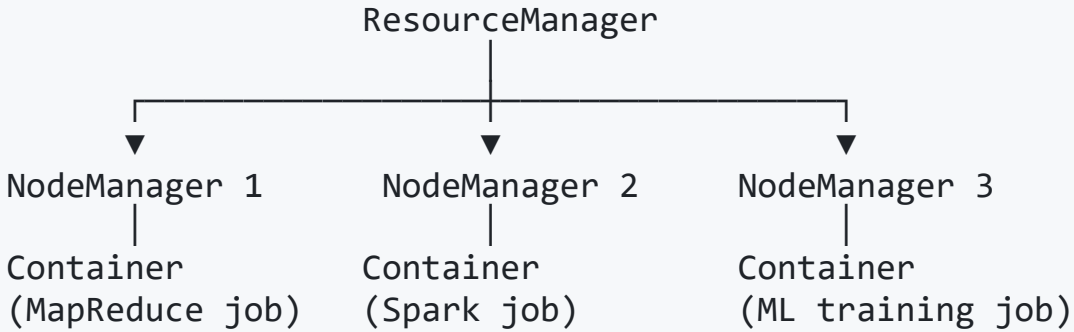
Pierwotny Hadoop (MapReduce v1) miał jeden komponent robiący **wszystko**: zarządzanie zasobami **i** planowanie zadań. To się nie skalowało.

YARN oddzielił te odpowiedzialności:

Komponent	Rola
ResourceManager	"Szef klastra" — globalnie zarządza zasobami (CPU, RAM) całego klastra
NodeManager	Działa na **każdej** maszynie — monitoruje lokalne zasoby, raportuje do ResourceManager
ApplicationMaster	Jeden na **każde uruchomione zadanie** — negocjuje zasoby, pilnuje wykonania
Container	Jednostka przydzielonych zasobów (np. "2 CPU + 4GB RAM") na której faktycznie działa kod

🎛️ YARN — dlaczego to było przełomowe

Dzięki YARN, klaster Hadoop mógł uruchamiać **nie tylko** MapReduce:



> 🎯 **YARN = system operacyjny klastra.** Zarządza zasobami niezależnie od tego, jaki framework przetwarzania działa na górze (MapReduce, Spark, Flink...).

To właśnie **dzięki YARN** Apache Spark mógł "wskoczyć" do istniejących klastrów Hadoop bez przebudowy całej infrastruktury!

🚀 Dlaczego Apache Spark zastąpił MapReduce?

Apache Spark rozwiązuje główny problem MapReduce: przetwarza dane **w pamięci RAM**, a nie na dysku między każdą fazą.

```
# MapReduce: zapis na dysk po KAŻDYM kroku
# Map → dysk → Shuffle → dysk → Reduce → dysk

# Spark: dane trzymane w pamięci przez CAŁE przetwarzanie
# (chyba że jest ich za dużo, żeby się zmieściły — wtedy "spill to disk")
```

Cecha	MapReduce	Spark
Przechowywanie danych pośrednich	Dysk	RAM (domyślnie)
Wydajność (zadania iteracyjne)	Wolne	**10-100× szybsze**
API	Tylko Java, niskopoziomowe	Python/Scala/SQL, wysokopoziomowe
Obsługa ML / grafów / streamingu	Trudna, "na siłę"	Wbudowane biblioteki

> 🔮 O Sparku szczegółowo porozmawiamy na **Wykładzie 4**!

🗺️ Ekosystem Hadoop — pozostałe narzędzia

Narzędzie	Do czego służy	Analogia
Hive	SQL-owy interfejs do danych w HDFS	"SQL nakładka na Hadoop"
HBase	Rozproszona baza NoSQL (kolumnowa) na HDFS	"Ogromny, szybki słownik klucz-wartość"
Sqoop	Import/export danych między HDFS a bazami SQL	"Most między światem SQL i Hadoop"
Oozie	Planowanie i orkiestracja workflowów Hadoop	"Cron dla zadań Big Data"
Pig	Wysokopoziomowy język skryptowy do MapReduce	"Skrypty zamiast pisania Javy"

> 💡 Nie musisz znać ich wszystkich na pamięć — ważne, żebyś **wiedział, że istnieją** i do czego służą, gdy zobaczysz je w ogłoszeniu o pracę!

🐝 Hive — SQL na HDFS

```sql
-- Wygląda jak zwykły SQL...
SELECT kategoria, SUM(przychod) AS suma
FROM sprzedaz
WHERE rok = 2026
GROUP BY kategoria
ORDER BY suma DESC;
```

Pod maską Hive **tłumaczy** ten SQL na zadania MapReduce (lub Spark!) i uruchamia je na danych w HDFS.

> 🎯 Idealne dla analityków, którzy znają SQL, ale nie chcą pisać Javy/Pythona do przetwarzania petabajtów danych.

⚠️ Częste pułapki — ekosystem Hadoop

Pułapka	Dlaczego to problem	Co zamiast tego
"Postawmy Hadoop, bo mamy dużo danych"	Setup i utrzymanie klastra to duży koszt	Najpierw sprawdź, czy naprawdę potrzebujesz rozproszenia (5V!)
Mylenie HDFS z bazą danych	HDFS to system **plików**, nie baza z transakcjami	Użyj Hive/HBase, jeśli potrzebujesz zapytań/transakcji
Ignorowanie replikacji przy planowaniu przestrzeni	100 TB danych = ~300 TB na dysku (replication factor 3)	Zawsze licz przestrzeń **razy 3**
Zbyt duża liczba małych zadań MapReduce	Narzut na uruchomienie zadania (JVM startup itd.)	Batch'uj zadania, rozważ Spark

📝 Kluczowe wnioski

1. ✅ **Big Data = 5V** (Volume, Velocity, Variety, Veracity, Value) — nie tylko "dużo danych"
2. ✅ **HDFS** dzieli pliki na bloki i replikuje je (domyślnie ×3) dla odporności na awarie
3. ✅ **MapReduce** = map (równolegle) → shuffle (grupowanie) → reduce (agregacja)
4. ✅ **YARN** oddzielił zarządzanie zasobami od logiki przetwarzania — otworzył drogę dla Sparka
5. ✅ **Spark** zastąpił MapReduce dzięki przetwarzaniu w pamięci RAM

🔮 Co dalej?

Wykład 4 (27.10.2026): Apache Spark — architektura, RDD i podstawy PySpark

Zobaczymy, jak te same idee (podział danych, przetwarzanie równoległe) wyglądają w praktyce w Sparku — tylko **dużo szybciej** i z dużo przyjemniejszym API! 🚀

Ćwiczenia (dziś, po wykładzie): Zbudujemy **własny mini-MapReduce w czystym Pythonie**, analizując 2000 syntetycznych recenzji filmowych — zliczymy najczęstsze słowa, porównamy pozytywne i negatywne recenzje, a nawet spróbujemy zrównoleglić nasze rozwiązanie!

❓ Pytania?

Dziękuję za uwagę! 🎉

Zobaczymy się na ćwiczeniach — czas na kod! 💻🐘